

1. Objectives:

1. To understand the basic concept of Binary Tree and Binary Search Tree (BST).
2. To implement preorder, inorder, postorder traversal in Tree.

2. Introduction:

- **Tree:** A Tree is a non-linear hierarchical data structure consisting of nodes connected by edges. The topmost node is called the root, and each node may contain child nodes. Trees are widely used in computer science for representing hierarchical relationships, efficient searching, and data organization. A Binary Tree is a special type of tree where each node can have at most two child nodes, known as the left child and right child. In this lab, Binary Tree operations such as node insertion, preorder traversal, inorder traversal, postorder traversal

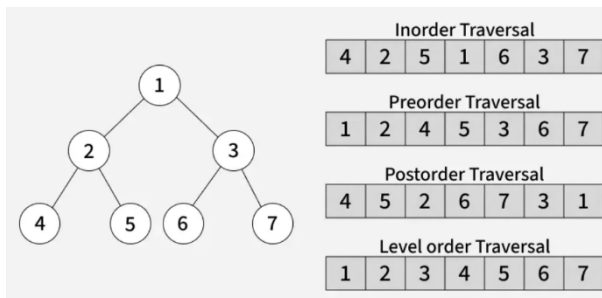


Fig 11.1: Tree

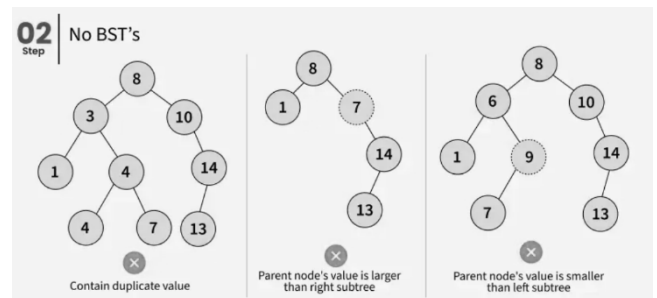


Fig 11.2: BST

- **BST:** Bread A Binary Search Tree (BST) is a special type of binary tree in which each node follows a specific ordering rule. In a BST, all values in the left subtree of a node are smaller than the node's value, and all values in the right subtree are greater than the node's value. This property is maintained for every node in the tree. Each node in a BST contains three parts: data, a pointer to the left child, and a pointer to the right child. The left child stores smaller values, while the right child stores larger values

3. Procedure

Experiment 1: Tree traversal, level order Height etc.

Input:

- value – Data for tree nodes
- root – Root node of tree
- node – Current node
- parent – Parent node

Output:

- Printed preorder, inorder, postorder, level order traversal
- Height of tree
- Width of tree
- Number of leaf and non-leaf nodes

Algorithm:

createNode(value)

1. create newNode
2. newNode.data $\leftarrow$ value
3. newNode.left $\leftarrow$ NULL
4. newNode.right $\leftarrow$ NULL
5. **return** newNode

creatRoot(value)

1. $\text{root} \leftarrow \text{createNode}(\text{value})$

insertLeft(parent, value)

1. $\text{parent.left} \leftarrow \text{createNode}(\text{value})$

insertRight(parent, value)

1. $\text{parent.right} \leftarrow \text{createNode}(\text{value})$

displayPre(node)

1. **if** $\text{node} == \text{NULL}$ **then**
2. **return**
3. print node.data
4. call $\text{displayPre}(\text{node.left})$
5. call $\text{displayPre}(\text{node.right})$

displayIno(node)

1. **if** $\text{node} == \text{NULL}$ **then**
2. **return**
3. call $\text{displayIno}(\text{node.left})$
4. print node.data
5. call $\text{displayIno}(\text{node.right})$

displayPost(node)

1. **if** $\text{node} == \text{NULL}$ **then**
2. **return**
3. call $\text{displayPost}(\text{node.left})$
4. call $\text{displayPost}(\text{node.right})$
5. print node.data

displayLevelOrder(start)

1. **if** $\text{start} == \text{NULL}$ **then**
2. **return**
3. create empty queue q
4. push start into q
5. **while** q is not empty **do**
6. $\text{current} \leftarrow$ front element of q
7. pop q
8. print current.data
9. **if** $\text{current.left} \neq \text{NULL}$ **then**
10. push current.left into q
11. **if** $\text{current.right} \neq \text{NULL}$ **then**
12. push current.right into q

getHeight(node)

1. **if** $\text{node} == \text{NULL}$ **then**
2. **return** 0
3. $\text{leftHeight} \leftarrow \text{getHeight}(\text{node.left})$
4. $\text{rightHeight} \leftarrow \text{getHeight}(\text{node.right})$
5. **return** $\max(\text{leftHeight}, \text{rightHeight}) + 1$

getWidth(node)

1. **if** $\text{node} == \text{NULL}$ **then**
2. **return** 0
3. create empty queue q
4. push node into q
5. $\text{maxWidth} \leftarrow 0$

6. **while** q is not empty do
7. levelSize $\leftarrow$ size of q
8. maxWidth $\leftarrow$ max(maxWidth, levelSize)
9. **for** i $\leftarrow$ 0 to levelSize-1 do
10. current $\leftarrow$ front element of q
11. pop q
12. **if** current.left $\neq$ NULL **then**
13. push current.left into q
14. **if** current.right $\neq$ NULL **then**
15. push current.right into q
16. **return** maxWidth

countLeafNodes(node)

1. **if** node == NULL **then**
2. **return** 0
3. **if** node.left == NULL AND node.right == NULL **then**
4. **return** 1
5. **return** countLeafNodes(node.left) + countLeafNodes(node.right)

countNonLeafNodes(node)

1. **if** node == NULL **then**
2. **return** 0
3. **if** node.left $\neq$ NULL OR node.right $\neq$ NULL **then**
4. **return** 1 + countNonLeafNodes(node.left) + countNonLeafNodes(node.right)
5. **return** 0

Experiment 2: BST, Max, Min

Input:

- value – Data to insert into BST
- root – Root node of BST
- node – Current node

Output:

- Printed preorder, inorder, postorder, level order traversal
- Maximum and minimum value of BST
- Search result ("Found" / "Not Found")

Algorithm:

insertValue(value)

1. root $\leftarrow$ insertNode(root, value)

insertNode(node, value)

1. **if** node == NULL **then**
2. **return** createNode(value)
3. **if** value < node.data **then**
4. node.left $\leftarrow$ insertNode(node.left, value)
5. **else if** value > node.data **then**
6. node.right $\leftarrow$ insertNode(node.right, value)
7. **return** node

searchValue(value)

1. **if** searchNode(root, value) == NULL **then**
2. **print** "Not Found"
3. **else**

4. print "Found"

searchNode(node, value)

1. **if** node == NULL **OR** node.data == value **then**
2. **return** node
3. **if** value < node.data **then**
4. **return** searchNode(node.left, value)
5. **return** searchNode(node.right, value)

getMax()

1. **if** root == NULL **then**
2. print "Tree is empty"
3. **return** -1
4. **return** findMax(root)

findMax(node)

1. **if** node.right == NULL **then**
2. **return** node.data
3. **return** findMax(node.right)

getMin()

1. **if** root == NULL **then**
2. print "Tree is empty"
3. **return** -1
4. **return** findMin(root)

findMin(node)

1. **if** node.left == NULL **then**
2. **return** node.data
3. **return** findMin(node.left)

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** Visual Studio Code 1.109

5. Experiment Implementation:

Experiment 1: Tree traversal, level order Hight etc.

```
#include <iostream>
#include <queue>
using namespace std;
class Trees{
public:
    struct Node{
        int data;
        Node *left;
        Node *right;
    };
    Node *root;
    Trees(){
        root = NULL;
    }
    Node *createNode(int value){
        Node *newNode = new Node();
        newNode->data = value;
        newNode->left = NULL;
        newNode->right = NULL;
        return newNode;
    }
    void creatRoot(int value){
        root = createNode(value);
    }
    void insertLeft(Node *parent, int value){
        parent->left = createNode(value);
    }
    void insertRight(Node *parent, int value){
```

```

        parent->right = createNode(value);
    }
    void displayPre(Node *node){
        if (node == NULL){
            return;
        }
        cout << node->data << " ";
        displayPre(node->left);
        displayPre(node->right);
    }
    void displayIno(Node *node){
        if (node == NULL)
            return;
        displayIno(node->left);
        cout << node->data << " ";
        displayIno(node->right);
    }
    void displayPost(Node *node){
        if (node == NULL)
            return;
        displayPost(node->left);
        displayPost(node->right);
        cout << node->data << " ";
    }
    void displayLevelOrder(Node *start){
        if (start == NULL)
            return;
        queue<Node *> q;
        q.push(start);
        while (!q.empty()){
            Node *current = q.front();
            q.pop();
            cout << current->data << " ";
            if (current->left != NULL){
                q.push(current->left);
            }
            if (current->right != NULL){
                q.push(current->right);
            }
        }
    }
    int getHeight(Node *node){
        if (node == NULL){
            return 0;
        }
        int leftHeight = getHeight(node->left);
        int rightHeight = getHeight(node->right);

        return max(leftHeight, rightHeight) + 1;
    }
    int getWidth(Node *node){
        if (node == NULL)
            return 0;
        queue<Node *> q;
        q.push(node);
        int maxWidth = 0;
        while (!q.empty()){
            int levelSize = q.size();
            maxWidth = max(maxWidth, levelSize);
            for (int i = 0; i < levelSize; i++){
                Node *current = q.front();
                q.pop();
                if (current->left != NULL)
                    q.push(current->left);
                if (current->right != NULL)
                    q.push(current->right);
            }
        }
        return maxWidth;
    }
    int countLeafNodes(Node *node){
        if (node == NULL)
            return 0;
        if (node->left == NULL && node->right == NULL)
            return 1;
        return countLeafNodes(node->left) + countLeafNodes(node->right);
    }
    int countNonLeafNodes(Node *node){
        if (node == NULL)

```

```

        return 0;
        if (node->left != NULL || node->right != NULL){
            return 1 + countNonLeafNodes(node->left) + countNonLeafNodes(node->right);
        }
        return 0;
    }
};

int main(){
    Trees t;
    t.creatRoot(1);
    t.insertLeft(t.root, 2);
    t.insertRight(t.root, 3);
    t.insertLeft(t.root->left, 4);
    t.insertRight(t.root->left, 5);
    t.insertLeft(t.root->right, 6);
    t.insertRight(t.root->right, 7);
    cout << "Preorder: ";
    t.displayPre(t.root);
    cout << endl;
    cout << "Inorder: ";
    t.displayIno(t.root);
    cout << endl;
    cout << "Postorder: ";
    t.displayPost(t.root);
    cout << endl;
    cout << "Level Order: ";
    t.displayLevelOrder(t.root);
    cout << endl;
    cout << "Height: " << t.getHeight(t.root) << endl;
    cout << "Width: " << t.getWidth(t.root) << endl;
    cout << "Leaf Nodes: " << t.countLeafNodes(t.root) << endl;
    cout << "Non-Leaf Nodes: " << t.countNonLeafNodes(t.root) << endl;
    return 0;
}

```

Table 11.1: Experiment 1 output

Input	Expected Output	Actual Output
	Preorder: 1 2 4 5 3 6 7 Inorder: 4 2 5 1 6 3 7 Postorder: 4 5 2 6 7 3 1 Level Order: 1 2 3 4 5 6 7 Height: 3 Width: 4 Leaf Nodes: 4 Non-Leaf Nodes: 3	Preorder: 1 2 4 5 3 6 7 Inorder: 4 2 5 1 6 3 7 Postorder: 4 5 2 6 7 3 1 Level Order: 1 2 3 4 5 6 7 Height: 3 Width: 4 Leaf Nodes: 4 Non-Leaf Nodes: 3

Experiment 2: BST, Max, Min

```

#include <iostream>
#include <queue>
using namespace std;
class BST{
public:
    struct Node{
        int data;
        Node *left;
        Node *right;
    };
    Node *root;
    BST(){
        root = NULL;
    }
    Node *createNode(int value){
        Node *newNode = new Node();
        newNode->data = value;
        newNode->left = NULL;
        newNode->right = NULL;
        return newNode;
    }
    void insertvalue(int value){
        root = insertNode(root, value);
    }
    Node *insertNode(Node *node, int value){
        if (node == NULL){
            return createNode(value);
        }
    }
}

```

```

        if (value < node->data){
            node->left = insertNode(node->left, value);
        }
        else if (value > node->data){
            node->right = insertNode(node->right, value);
        }
        return node;
    }
void displayPre(Node *node){
    if (node == NULL){
        return;
    }
    cout << node->data << " ";
    displayPre(node->left);
    displayPre(node->right);
}
void displayIno(Node *node){
    if (node == NULL)
        return;
    displayIno(node->left);
    cout << node->data << " ";
    displayIno(node->right);
}
void displayPost(Node *node){
    if (node == NULL)
        return;
    displayPost(node->left);
    displayPost(node->right);
    cout << node->data << " ";
}
void displayLevelOrder(Node *start){
    if (start == NULL)
        return;
    queue<Node *> q;
    q.push(start);
    while (!q.empty()){
        Node *current = q.front();
        q.pop();
        cout << current->data << " ";
        if (current->left != NULL){
            q.push(current->left);
        }
        if (current->right != NULL){
            q.push(current->right);
        }
    }
}
void searchValue(int value){
    if (searchNode(root, value) == NULL){
        cout << "Not Found" << endl;
    }
    else{
        cout << "Found" << endl;
    }
}
Node *searchNode(Node *node, int value){
    if (node == NULL || node->data == value){
        return node;
    }
    if (value < node->data){
        return searchNode(node->left, value);
    }
    return searchNode(node->right, value);
}
int getMax(){
    if (root == NULL){
        cout << "Tree is empty" << endl;
        return -1;
    }
    return findMax(root);
}
int findMax(Node *node){
    if (node->right == NULL){
        return node->data;
    }
    return findMax(node->right);
}
int getMin(){

```

```

        if (root == NULL){
            cout << "Tree is empty" << endl;
            return -1;
        }
        return findMin(root);
    }
    int findMin(Node *node){
        if (node->left == NULL){
            return node->data;
        }
        return findMin(node->left);
    }
};

int main(){
    BST b;
    b.insertvalue(50);
    b.insertvalue(30);
    b.insertvalue(70);
    b.insertvalue(20);
    b.insertvalue(40);
    b.insertvalue(60);
    b.insertvalue(80);
    cout << "BST: ";
    b.displayIno(b.root);
    cout << endl;
    cout << "Max: " << b.getMax() << endl;
    cout << "Min: " << b.getMin() << endl;
    return 0;
}

```

Table 11.2: Experiment 2 output

Input	Expected Output	Actual Output
	BST: 20 30 40 50 60 70 80 Max: 80 Min: 20	BST: 20 30 40 50 60 70 80 Max: 80 Min: 20

6. Discussion & Conclusion:

In this lab, we learned Binary Tree and Binary Search Tree. Different traversal techniques such as preorder, inorder, postorder, and level order traversal were implemented. The Binary Tree implementation demonstrated operations like height calculation, width calculation, and counting leaf and non-leaf nodes, while the BST implementation showed how ordered insertion improves searching efficiency. Recursive functions played a major role in traversal and searching operations, and queue was used for level order traversal. Some challenges were faced in understanding recursive function flow and maintaining correct node connections during insertion. Logical tracing of recursion and tree structure was more difficult than syntax-related issues.

7. Reference:

1. <https://www.geeksforgeeks.org/dsa/tree-traversals-inorder-preorder-and-postorder/> accessed at 7.45 PM on 15th May 2026.
2. <https://www.geeksforgeeks.org/dsa/binary-search-tree-data-structure/> accessed at 08:25 Pm on 15th May 2026.